PROGRAM-2

1. Problem Analysis

- **Input:** Household ID, monthly electricity consumption in units (kWh), fixed charge, surcharge rate, rebate rate, and previous monthly bills.
- **Processing:** Calculate slab-wise charges, add fixed charges, apply surcharges and rebates, compare monthly bills, and identify unusual increases.
- **Output:** Monthly electricity bill, bill comparison report, and alerts for unusually high increases.

Log inSign up for free

1. **You said:**

computational thinking, algorithms, flowcharts, pseudocode, variables, data types, operators, conditional statements, loops, strings, pattern generation, lists, nested lists, tuples, tuple operations, and unpacking. 1. University Scholarship Allocation: Design a system that evaluates applicants using aggregate marks, subject-wise minimum scores, attendance, and income criteria, ranks eligible students, and resolves ties using predefined rules.

2. **ChatGPT said:**

1. University Scholarship Allocation System

This problem applies computational thinking, algorithms, flowcharts, pseudocode, variables, data types, operators, conditional statements, loops, strings, lists, nested lists, tuples, tuple operations, and unpacking to a real-world scholarship allocation system.

1. Problem statement

Design a program that evaluates university scholarship applicants using the following criteria:

- Aggregate marks must meet the minimum requirement.
- Each subject score must meet its minimum requirement.
- Attendance must meet the required percentage.
- Family annual income must be within the specified limit.
- Eligible students must be ranked according to their performance.
- Ties must be resolved using predefined rules.

2. Eligibility criteria

For this example, assume the university uses the following rules.

Criterion	**Requirement**
Aggregate marks	At least 75%

Mathematics	At least 60 marks
Physics	At least 50 marks
Chemistry	At least 50 marks
Attendance	At least 80%
Annual family income	At most ₹3,00,000

Tie-breaking rules:

7. Higher aggregate marks rank first.

8. If aggregate marks are equal, higher attendance ranks first.

9. If attendance is also equal, lower annual family income ranks first.

10. If all three are equal, alphabetical order of the student's name decides the rank.

These are illustrative rules; an actual university should use its officially approved criteria.

3. Computational thinking

The problem can be divided into four stages.

- **Decomposition:** Separate applicant data collection, eligibility checking, ranking, and scholarship allocation.
- **Pattern recognition:** Identify common eligibility conditions and repeated comparisons between applicants.
- **Abstraction:** Focus on marks, attendance, income, and name while ignoring irrelevant information.
- **Algorithm design:** Define a sequence of steps to check eligibility and sort qualified applicants.

4. Algorithm

15. Start.

16. Read the number of applicants.

17. Store each applicant's name, aggregate marks, subject marks, attendance, and annual family income.

18. Check all eligibility conditions.

19. If every condition is satisfied, add the applicant to the eligible list.

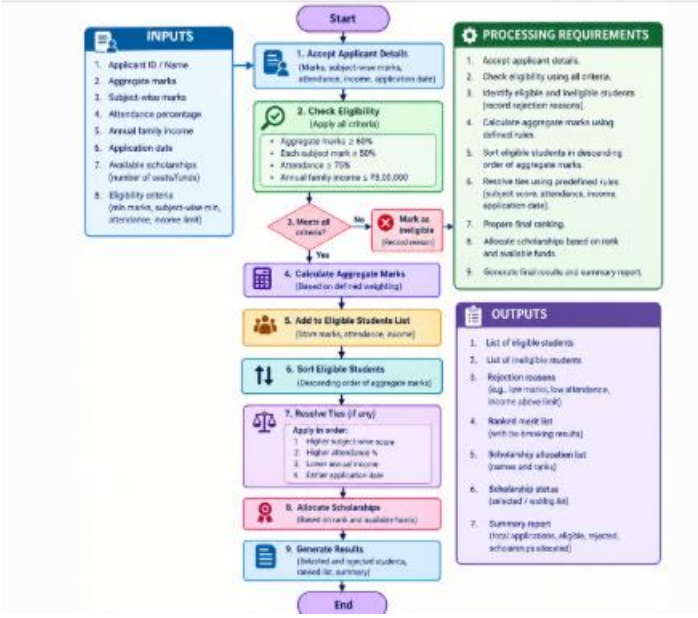
20. Sort eligible applicants by aggregate marks in descending order.

21. Resolve ties using attendance in descending order, income in ascending order, and name in alphabetical order.

22. Display the ranked eligible applicants.

23. Stop.

1. Flowchart



Start

Input applicant details

Check marks, subjects, attendance and income

All conditions satisfied?

Yes

Add to eligible list

No

Reject applicant

Repeat for all applicants

Sort eligible applicants using tie-breaking rules

Display scholarship ranking

Stop

Conceptual flowchart of the scholarship allocation process.

6. Pseudocode

text

BEGIN

```
eligible_students = empty list

INPUT number_of_applicants

FOR each applicant:
    INPUT name
    INPUT aggregate_marks
    INPUT mathematics_marks
    INPUT physics_marks
    INPUT chemistry_marks
    INPUT attendance
    INPUT annual_income

    IF aggregate_marks >= 75 AND
       mathematics_marks >= 60 AND
       physics_marks >= 50 AND
       chemistry_marks >= 50 AND
       attendance >= 80 AND
       annual_income <= 300000 THEN

        ADD applicant to eligible_students

    END IF
END FOR

SORT eligible_students BY:
    aggregate_marks descending
    attendance descending
    annual_income ascending
    name ascending
```

```
    DISPLAY eligible_students with ranks

END
```

7. Python implementation

The following program demonstrates all the main concepts in the question.

python

```
# University Scholarship Allocation System

MIN_AGGREGATE = 75
MIN_MATHS = 60
MIN_PHYSICS = 50
MIN_CHEMISTRY = 50
MIN_ATTENDANCE = 80
MAX_INCOME = 300000

# Nested list: each applicant is represented by a list
applicants = [
    ["Ananya", 90, 85, 80, 82, 95, 200000],
    ["Bharat", 90, 85, 80, 82, 90, 180000],
    ["Charan", 78, 65, 55, 60, 85, 250000],
    ["Divya", 88, 70, 65, 68, 92, 350000],
    ["Esha", 82, 55, 70, 65, 90, 150000],
    ["Farhan", 85, 75, 70, 72, 88, 220000],
]

eligible = []
rejected = []

# Check eligibility using a loop and conditional statements
```

```
for student in applicants:
    name, aggregate, maths, physics, chemistry, attendance, income = student

    if (aggregate >= MIN_AGGREGATE
            and maths >= MIN_MATHS
            and physics >= MIN_PHYSICS
            and chemistry >= MIN_CHEMISTRY
            and attendance >= MIN_ATTENDANCE
            and income <= MAX_INCOME):

        eligible.append(student)
    else:
        rejected.append(name)

# Tuple operations and sorting
# Sort by aggregate descending, attendance descending,
# income ascending, and name alphabetically.
eligible.sort(key=lambda s: (-s[1], -s[5], s[6], s[0]))

# Display results using string formatting
print("SCHOLARSHIP RANK LIST")
print("-" * 75)

for rank, student in enumerate(eligible, start=1):
    name, aggregate, maths, physics, chemistry, attendance, income = student

    print(
        f"Rank {rank}: {name} | "
        f"Aggregate: {aggregate}% | "
        f"Attendance: {attendance}% | "
        f"Income: Rs. {income}"
```

)

print("\nRejected applicants:", ", ".join(rejected))

8. Expected output

text

SCHOLARSHIP RANK LIST

Rank 1: Ananya | Aggregate: 90% | Attendance: 95% | Income: Rs. 200000

Rank 2: Bharat | Aggregate: 90% | Attendance: 90% | Income: Rs. 180000

Rank 3: Farhan | Aggregate: 85% | Attendance: 88% | Income: Rs. 220000

Rank 4: Charan | Aggregate: 78% | Attendance: 85% | Income: Rs. 250000

Rejected applicants: Divya, Esha

Divya is rejected because her income exceeds the limit. Esha is rejected because her Mathematics score is below the minimum.

Ananya ranks above Bharat because their aggregate marks are equal, but Ananya has higher attendance. Attendance takes priority over income under the predefined rules.

9. Programming concepts used

Concept	**Application in the program**
Variables	aggregate, attendance, income
Data types	Strings for names, integers for marks and income
Operators	>=, <=, and, unary -
Conditional statements	if and else for eligibility
Loops	for to examine each applicant
Strings	Names and formatted output
Pattern generation	"-" * 75 creates a separator

Lists	eligible, rejected
Nested lists	applicants stores multiple applicant records
Tuples	The unpacked applicant values can be grouped into a tuple; the sort key is also a tuple
Tuple operations	Comparing multi-part sort keys lexicographically
Unpacking	name, aggregate, maths, ... = student
Algorithm	Eligibility checking followed by sorting
Computational thinking	Breaking the problem into manageable stages

Note on tuples: Each applicant is stored as a list so that the record can be modified if necessary. A tuple could instead be used for immutable applicant records. Python's sorting key returns a tuple, and Python compares its elements from left to right to apply the tie-breaking rules.

10. Time complexity

Let $n$be the number of applicants.

- Eligibility checking takes $O(n)$time.
- Sorting eligible applicants takes $O(n \log n)$time in the worst case.
- Overall time complexity is $O(n \log n)$.
- Storing applicant records requires $O(n)$space.

11. Conclusion

The University Scholarship Allocation System demonstrates how computational thinking can solve a practical decision-making problem. It checks eligibility using multiple conditions, stores records in nested lists, unpacks applicant details, ranks eligible students using tuple-based sorting, and applies consistent tie-breaking rules. The result is a transparent and repeatable scholarship ranking process.

3. **You said:**
4. PROBLEM SOLVING APPROACH
5. **Problem-Solving Approach: University Scholarship Allocation System**

The problem-solving approach explains how to analyze the scholarship allocation problem and develop a step-by-step solution using computational thinking.

1. Problem Definition

Develop a system to identify eligible students for university scholarships based on aggregate marks, subject-wise minimum marks, attendance, and annual family income. Rank eligible students and resolve ties using predefined rules.

2. Problem Analysis

Identify the inputs, processing requirements, and outputs.

- **Input:** Student name, aggregate marks, subject-wise marks, attendance percentage, and annual family income.
- **Processing:** Check eligibility, identify rejected applicants, rank eligible students, and resolve ties.
- **Output:** List of eligible students in rank order and list of rejected applicants.

3. Identify the Constraints

Criterion	**Minimum/Maximum requirement**
Aggregate marks	Minimum 75%
Mathematics	Minimum 60 marks
Physics	Minimum 50 marks
Chemistry	Minimum 50 marks
Attendance	Minimum 80%
Annual family income	Maximum ₹3,00,000

4. Design the Algorithm

4. Start the program.

5. Read the applicant details.

6. Check whether the applicant satisfies all eligibility conditions.

7. If all conditions are satisfied, add the applicant to the eligible list; otherwise, add the applicant to the rejected list.

8. Repeat the process for all applicants.

9. Sort eligible students by:

- Aggregate marks in descending order.
- Attendance in descending order.
- Annual family income in ascending order.

- Name in alphabetical order.

10. Assign ranks and display the results.

11. Stop the program.

5. Develop the Solution

Write a Python program using variables, lists, conditional statements, loops, and sorting operations to implement the algorithm.

6. Testing and Verification

Test the program with different cases:

- A student who satisfies every condition.
- A student whose aggregate marks are below the minimum.
- A student who fails one subject-wise requirement.
- A student whose attendance is below 80%.
- A student whose family income exceeds the limit.
- Two students with equal aggregate marks to verify tie-breaking.

7. Evaluate the Results

Verify that only eligible students appear in the ranking, rejected applicants are correctly identified, and all ties are resolved according to the predefined rules.

8. Conclusion

The problem-solving approach converts a real-world scholarship allocation problem into a structured computational solution. By defining the problem, analyzing requirements, designing an algorithm, implementing the program, and testing its results, the system ensures fair, consistent, and transparent scholarship allocation.

Log in for personalized, step-by-step guidance.

Log inSign up for free

6. **You said:**

2. Progressive Electricity Billing: Calculate household bills using multiple consumption slabs, fixed charges, surcharges, and rebates, then compare bills across several months and identify unusual increases.

7. **ChatGPT said:**

2. Progressive Electricity Billing System — Problem-Solving Approach

1. Problem Definition

Develop an electricity billing system that calculates household electricity bills based on progressive consumption slabs, fixed charges, surcharges, and rebates. The system should compare bills across multiple months and identify unusual increases in electricity expenses.

2. Problem Analysis

Identify the inputs, processing requirements, and outputs.

- **Input:** Household ID, monthly electricity consumption in units (kWh), fixed charge, surcharge rate, rebate rate, and previous monthly bills.
- **Processing:** Calculate slab-wise charges, add fixed charges, apply surcharges and rebates, compare monthly bills, and identify unusual increases.
- **Output:** Monthly electricity bill, bill comparison report, and alerts for unusually high increases.

3. Define the Billing Rules

Assume the following illustrative slab rates. Actual electricity tariffs depend on the applicable utility and consumer category.

Consumption slab	**Rate per unit**
First 100 units	₹2.00
Next 100 units	₹3.00
Next 100 units	₹5.00
Above 300 units	₹7.00

Additional rules:

- Fixed charge: ₹100 per month.
- Surcharge: 5% of the energy charge.
- Rebate: 2% of the energy charge for eligible households.
- Unusual increase: The total bill increases by more than 30% compared with the previous month, provided the previous bill is greater than zero.

4. Computational Thinking

- **Decomposition:** Divide the problem into consumption calculation, bill calculation, monthly comparison, and anomaly detection.
- **Pattern recognition:** Identify recurring monthly consumption patterns and sudden bill increases.
- **Abstraction:** Focus on units consumed, slab rates, charges, rebates, and monthly totals.
- **Algorithm design:** Apply progressive rates in order, calculate the final bill, and compare it with previous bills.

5. Algorithm

1. Start.
2. Read the household details and monthly electricity consumption.
3. Calculate the energy charge using progressive slabs.
4. Add the fixed charge.
5. Calculate and add the surcharge.
6. If the household qualifies, calculate and subtract the rebate.
7. Store the final bill in a list.
8. Compare each month's bill with the previous month's bill.
9. If the increase exceeds 30%, display an unusual-increase alert.
10. Display monthly bills and comparison results.
11. Stop.

6.Flowchart

7. Python Program

python

```
# Progressive Electricity Billing System

SLAB_RATES = [2, 3, 5, 7]
SLAB_LIMITS = [100, 200, 300]
FIXED_CHARGE = 100
SURCHARGE_RATE = 0.05
REBATE_RATE = 0.02
INCREASE_LIMIT = 30

# Nested list: [month, units_consumed, rebate_eligible]
monthly_data = [
    ["January", 80, True],
    ["February", 120, True],
    ["March", 150, True],
    ["April", 240, False],
    ["May", 160, True],
```

```
]

bills = []

def calculate_energy_charge(units):
    charge = 0
    previous_limit = 0

    for i, rate in enumerate(SLAB_RATES):
        if i < len(SLAB_LIMITS):
            slab_units = min(
                max(units - previous_limit, 0),
                SLAB_LIMITS[i] - previous_limit
            )
        else:
            slab_units = max(units - previous_limit, 0)

        charge += slab_units * rate

        if i < len(SLAB_LIMITS):
            previous_limit = SLAB_LIMITS[i]

    return charge


# Calculate each monthly bill
for month, units, rebate_eligible in monthly_data:
    energy_charge = calculate_energy_charge(units)

    fixed_charge = FIXED_CHARGE
    surcharge = energy_charge * SURCHARGE_RATE
```

```
    rebate = energy_charge * REBATE_RATE if rebate_eligible else 0

    total_bill = energy_charge + fixed_charge + surcharge - rebate
    total_bill = round(total_bill, 2)

    bills.append([month, units, total_bill])

# Compare monthly bills
print("MONTHLY ELECTRICITY BILL REPORT")
print("-" * 65)

for i, (month, units, bill) in enumerate(bills):
    print(f"{month}: Units = {units}, Bill = Rs. {bill:.2f}")

    if i > 0:
        previous_bill = bills[i - 1][2]

        if previous_bill > 0:
            increase = ((bill - previous_bill) / previous_bill) * 100

            if increase > INCREASE_LIMIT:
                print(f"  ALERT: Bill increased by {increase:.2f}%")
            else:
                print(f"  Change: {increase:.2f}%")

print("-" * 65)
```

8. Expected Output

text

```
MONTHLY ELECTRICITY BILL REPORT
-----------------------------------------------------------------
January: Units = 80, Bill = Rs. 265.20
```

February: Units = 120, Bill = Rs. 386.20

ALERT: Bill increased by 45.66%

March: Units = 150, Bill = Rs. 477.00

Change: 23.51%

April: Units = 240, Bill = Rs. 994.00

ALERT: Bill increased by 108.39%

May: Units = 160, Bill = Rs. 505.00

Change: -49.20%